* Declare string

String name = "sahali" ;

name → "sahali" — contains char

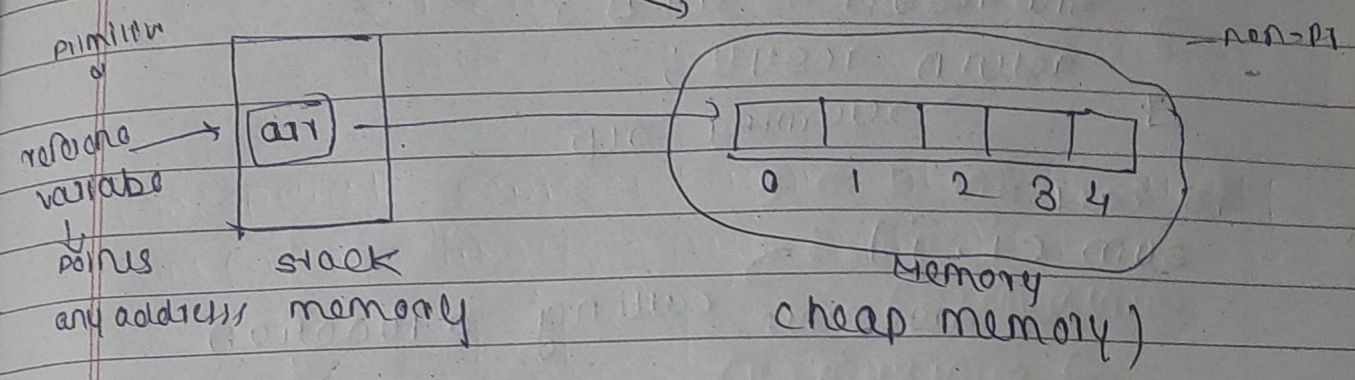
s.o. println (" ") ;
string type

To concatinate string we use + operator

27/8/26

* Memory

int [] arr = new int [5] ;



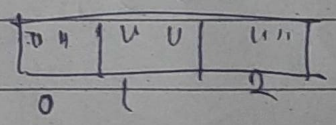
- int x = 4 — primitive datatype
  means x directly holds value 4

- when arr, it doesn't directly holds the array
  It is a reference to an array

* Strings in array

String [] name = new String [3] ;

→ name



```
// IP, NO OIP
static void sayHello(string name)
{
    s.o.pln("Hello" + name);
}

// NO IP, OP
static int getNumber() {
    return 10
}

// IP, OP
static int multiply(int a, int b)
{
    return (a * b);
}
```

* Function overloading

Function name same but diff parameters

* Function chaining

```
void main(——) {
{ A();
}
  void A() {
      B();
  }
  void B() {
      s.o.p("Hell");
  }
```

* Scope of variable

Scope of variable remains within the block only where it is declared.

27/8/26

* Java Functions | Recursions | Method overloading

* Functions in Java
  ↳ Block of code that can be reused

eg:

```
Returntype funname (int a, int b)
{
    // block of code
}
```

parameters

```
Public static int sum (int a, int b)
{
    int result = a+b;
    return result;
}
```

returning o/p

To call

```
sum (i, j);
```

Arguments calling a function

Types of Functions
- ↳ No i/p, No o/p
- ↳ I/P, No o/p
- ↳ No I/P, o/P
- ↳ I/P, o/P

eg:

```
// NO I/P, NO O/P
static void greet () {
    S.O.P/n ("Hello");
    return;
}
```

1) local scope - within block
2) global scope - outside the fun, inside class

* Recursion : - used in Tree, graphs, DP
  - A function which calls itself

```
void PrinNum (int n)
{
    if (n==0)
    {
        return;
    }
    PrinNum (n-1);
    c.o.p(n);
}
```

→ 5

- Base case
  - To stop recursion

→ recursive call

For eg PrinN 1 to 5

→ ~~5~~ 4 3

PrinNum (5)
↳ PrinNum (4)
↳ PrinNum (3)
↳ PrinNum (2)
↳ PrinNum (1)
↳ PrinNum (0)

O/P → 1
2
3
4
5

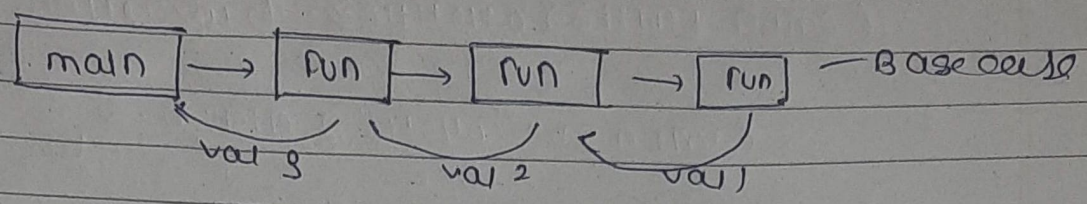
Fibonacci series
1, 1, 2, 3, 5, 8, ... n. nth fibo find.

x y

Fib(n) = Fib(n-1) + Fib(n-2);

Page No.
Date

Diagram representation



Recursion is a method of solving a computational problem where the solution depends on solution to smaller instances of same problem.

① Base case
② what to do
② inner run call

Q. Print numbers from n to 1 (decreasing order)